

خطة تطوير شاملة لنظام Nabil Gold

خارطة طريق عملية لتحويل النظام من Consensus Engine إلى Stateful Setup Engine

إعداد: Arena.ai Agent Mode — تاريخ الإصدار: 15-07-2026

الهدف	رفع دقة النظام، تحسين مكان الدخول، وبناء منطق Setup lifecycle
النطاق	SMC + execution + learning أولاً مع التركيز على XAU/USD
الأولوية	التحول من إجماع عام إلى استراتيجيات حسب نوع الفرصة

(1) الملخص التنفيذي

إلى محرك setups ذكي، يحتفظ بالقصة السعريّة، وينفذ من مناطق حقيقية، ويتعلم حسب نوع الفرصة والسياق. هذه الوثيقة تضع خطة تطوير كاملة لنظام Nabil Gold بهدف تحويله من محرك إجماع عام وقادر على التقاط الفرص التي يراها المحلل الجيد من نوع sweep → displacement → POI → mitigation. كل المحللين في كل الظروف، بل بناء نظام أقوى من أغلب المحللين العاديين، وأكثر اتساقاً من التداول اليدوي، الهدف الواقعي ليس التفوق الحرفي على

(2) التشخيص الحالي

ما الذي يجيده النظام اليوم؟

- بنية منظمة متعددة الوكلاء مع فصل واضح بين agents و services و scripts.
- إدارة مخاطر وفلاتر تشغيلية جيدة نسبياً مقارنة بكثير من المشاريع المماثلة.
- حفظ الحالة والصفقات والتقارير ووجود قاعدة بيانات ولوحة معلومات وتكامل Telegram.
- وجود أساس تقني للتعلم والتحليلات اللاحقة والتوسع المستقبلي.

أين الضعف الحقيقي؟

- النظام يفكر غالباً كـ Snapshot Engine وليس Setup Lifecycle Engine.
- طبقة التنفيذ ما زالت تميل إلى MARKET أكثر من zone/pending execution الحقيقي.
- منطق SMC موجود لكنه ليس قائد القرار في ظروفه الطبيعية.
- عتبات القرار متقاربة جداً بين أنواع setups المختلفة.
- التعلم الحالي مفيد لكنه ليس Context-aware بما يكفي.
- الإدارة بعد الدخول جيدة عموماً لكنها ليست Structure-aware حسب نوع setup.

(3) الرؤية المعمارية الجديدة

التحول المطلوب ليس مجرد تعديل thresholds، بل إعادة بناء عقل النظام على ست طبقات مترابطة:

- Data + Observability
- Setup Detection
- Setup State Machine
- Strategy-specific Decisioning
- Structure-aware Execution & Trade Management
- Regime-aware Learning + Analyst Distillation

(4) المرحلة 0 – تثبيت خط الأساس Baseline

المدة المقترحة: 3-5 أيام

الهدف

فهم أين يخسر النظام وأين يتأخر وما هي setups التي يفوتها قبل تعديل أي منطق.

المطلوب

و missed setup rate و late entry distance و entry efficiency و % RR capture و avg R achieved و إضافة مقاييس مثل expectancy

- miss_reason, displacement_score, sweep_side, poi_type, setup_quality, lead_agent, setup_type توسيع trade snapshot ليحفظ
- إظهار breakdowns حسب setup/session/regime/lead-agent في dashboard والتقارير.
- توسيع backtesting والتقارير حتى تعطي تحليلاً سببياً لا مجرد رقم PnL نهائي.

الملفات ذات الأولوية

```
services/database.py
supabase_schema_unified.sql
services/performance_dashboard.py
services/backtesting.py
scripts/run_backtest.py
dashboard/ و scripts/generate_dashboard.py
```

مقياس النجاح

تفشّل، وأيّ sessions أفضل، وهل MARKET entries أسوأ من zone entries، وأين يتأخّر النظام عن المحلل. لوحة تقول بوضوح أيّ setups

(5) المرحلة 1 – إصلاح طبقة الدخول Execution Layer

المدة: 1-2 أسبوع

الهدف

الانتقال من تنفيذ market-heavy إلى zone/pending execution حقيقي.

الأعمال المطلوبة

- إعادة تفعيل execution profile حقيقي عبر smart أو hybrid أو profile جديد مثل poi_pending.
- إلغاء أثر pending_support_removed: true أو إزالته من الإعدادات.
- ليقرر بوضوح بين MARKET وLIMIT وSTOP حسب نوع setup وقرب السعر من المنطقة.
- تطوير smart_entry داخل RiskManagementAgent
- توحيد شكل zone object بحيث يحتوي proximal وdistal وfill_rule وinvalidation anchor وsource.
- حفظ إشارات pending في run_analysis مع replace/cancel للإشارات القديمة لنفس setup.
- Telegram message لإظهار نوع الدخول والمنطقة وال invalidation والهدف السعري المرتبط بالسيولة.
- تطوير

الملفات ذات الأولوية

```
config.json
agents/risk_management_agent.py
agents/open_trades_manager.py
scripts/run_analysis.py
services/database.py
services/telegram_bot.py
```

مقياس النجاح

تحسن ملحوظ في entry efficiency وplanned RR وتقليل الصفقات التي تدخل بعد ضياع الميزة الأصلية.

(6) المرحلة 2 – بناء Setup State Machine

المدة: 2-3 أسابيع

الهدف

تحويل النظام من قراءة لحظية إلى منطق يحتفظ بالقصة السعرية عبر الزمن.

الحالات المقترحة

- DETECTED
- SWEEP_CONFIRMED
- DISPLACEMENT_CONFIRMED
- MSS_CONFIRMED
- POI_MARKED
- MITIGATION_PENDING
- ENTRY_ARMED
- ENTRY_TRIGGERED
- INVALIDATED
- EXPIRED

تطوير SMC Agent

- إخراج sweep side و sweep quality و equal highs/lows و session high/low raid.
- قياس displacement strength و MSS/CHOCH quality و order block rank و FVG rank.
- تحديد premium/discount score و target liquidity map و mitigation status.

تطوير run_analysis وقاعدة البيانات

- تحديث setup state مع كل دورة بدل انتظار إجماع لحظي فقط.
- إضافة setup_id ثابت لكل سيناريو سعري.
- إضافة جداول setup_candidates و setup_state_events.

الملفات ذات الأولوية

```
agents/smc_agent.py
scripts/run_analysis.py
services/database.py
services/setup_memory.py (حرقتم ديدج فلم)
supabase_schema_unified.sql
```

(7) المرحلة 3 — قرار مختلف لكل نوع Setup

المدة: 2 أسبوع

الهدف

عدم استخدام نفس منطق القرار لكل أنواع الصفقات.

الاستراتيجيات الأساسية المقترحة

- liquidity_reversal
- trend_pullback
- range_fade

لكل profile يجب تحديد

- lead agent و supporting agents
- soft veto و hard veto
- min agents و min confidence
- نوع التنفيذ المناسب
- session preference و daily bias requirement
- الملائم بعد الدخول management profile

الملفات ذات الأولوية

```
agents/decision_agent.py
config.json
services/strategy_profiles.py (حرتقم ديدج فلم)
```

— إدارة المخاطر والإدارة بعد الدخول بشكل Structure-aware (8 المرحلة 4)

المدة: 2 أسبوع

الهدف

تخصيص الوقف والأهداف والإدارة حسب نوع setup والبنية السعرية.

التعديلات المطلوبة

- في SL reversal setup خلف sweep extreme أو distal edge أو invalidation candle.
- في SL continuation setup خلف pullback structure أو continuation OB.
- ربط الأهداف بالسيولة الداخلية والخارجية وليس ATR فقط.
- إضافة management profiles منفصلة داخل OpenTradesManager.
- حفظ management_profile و invalidation_type و target_map مع كل صفقة.
- تقييد scale-in مؤقتا على continuation setups فقط أو تعطيله حتى يثبت الأساس.

الملفات ذات الأولوية

```
agents/risk_management_agent.py
agents/open_trades_manager.py
services/database.py
config.json
services/telegram_bot.py
```

5 — تعلم حقيقي حسب السياق Regime-aware Learning (9 المرحلة)

المدة: 2-3 أسابيع

الهدف

تحويل التعلم من توصيات عامة إلى تعلم يفرق بين setup type و session و regime و lead agent.

التعديلات المطلوبة

- بناء contextual weight matrix حسب setup type و session و regime و daily bias alignment.
- جعل DecisionAgent يقرأ weight profile مناسب بدل أوزان ثابتة دائماً.
- تفعيل dynamic risk لاحقاً بشكل contextual وليس global فقط.
- إظهار نتائج التعلم الجديدة في dashboard والتقارير اليومية والأسبوعية.

الملفات ذات الأولوية

```
services/learning_service.py
agents/decision_agent.py
config.json
services/performance_dashboard.py
```

المرحلة 6 — استنساخ Edge المحلل Analyst Distillation (10)

المدة: 3-4 أسابيع ثم تستمر بشكل دائم

الهدف

تحويل خبرة المحلل اليدوي الأفضل لديك إلى features وقواعد ومقاييس قابلة للاختبار.

الأعمال المطلوبة

- بناء dataset باسم analyst_labels يحتوي timestamp و bias و setup_type و sweep_side و poi_type و poi_quality و intended_entry و targets و trade/no-trade.
- بناء أداة مقارنة bot vs analyst على نفس الأيام والشارات.
- على sweep quality و displacement strength و POI freshness و HTF alignment و session quality و Analyst Quality Scorer يعتمد.
- تغذية dashboard بمقاييس overlap و missed setups و false setups و entry lag و RR difference.

الملفات المقترحة

```
services/analyst_distillation.py
scripts/compare_analyst_vs_bot.py
services/database.py
dashboard/ sections ةنراقم لل ةديج
```

(11) المرحلة 7 — تحسين الدقة التنفيذية Micro-Precision

المدة: اختيارية بعد المراحل السابقة

الهدف

- رفع الدقة في التنفيذ حتى يقترب النظام من قراءة microstructure البشرية.
- إضافة M1/M3 للتنفيذ فقط مع إبقاء التحليل الأعلى على 5m/15m/1H/4H.
- session مثل London open sweep و NY open sweep و lunch drift و pre-news fake move.
- إضافة منطق microstructure.
- إضافة مصدر بيانات أدق أو fallback أقوى للتنفيذ عندما تسمح البنية بذلك.

(12) الترتيب الزمني المقترح

الشهر الأول

- المرحلة 0 كاملة
- المرحلة 1 كاملة
- بداية المرحلة 2

الشهر الثاني

- إكمال المرحلة 2
- المرحلة 3
- جزء من المرحلة 4

الشهر الثالث

- إكمال المرحلة 4
- المرحلة 5
- بداية المرحلة 6

(13) الملفات الأعلى أولوية

أولوية قصوى

```
scripts/run_analysis.py
agents/smc_agent.py
agents/decision_agent.py
agents/risk_management_agent.py
agents/open_trades_manager.py
services/database.py
config.json
```

أولوية ثانية

```
services/learning_service.py
services/performance_dashboard.py
services/telegram_bot.py
dashboard/*
```

(14) أول Sprint مقترح

- إضافة setup_type و lead_agent و setup_quality إلى snapshot الصفقة.
- إعادة تفعيل pending/zone execution الحقيقي.
- تطوير Telegram message لإظهار zone و invalidation و target liquidity.
- إضافة جدول setup_candidates.
- توسيع smc_agent لإخراج sweep/displacement/POI structure.
- بناء state machine أولي.
- فصل 3 strategy profiles.
- ربط thresholds بال profile.
- إضافة تقارير setup/session/regime by.
- قياس الفرق بين MARKET و ZONE entries.

(15) مؤشرات الأداء الرئيسية KPI

- Win rate
- Expectancy
- Avg R achieved
- RR capture %
- Entry efficiency
- Late entry distance
- Missed setup rate
- Setup completion rate
- Setup invalidation rate
- Performance by session
- Performance by regime
- Performance by setup family
- Performance by lead agent

(16) ما الذي لا يجب فعله الآن؟

- جديدة بكثرة؛ المشكلة الأساسية ليست نقص المؤشرات بل نقص فهم setup lifecycle والتنفيذ من المنطقة.
- لا تضيف مؤشرات
- لا تجعل Gemini صاحب القرار النهائي؛ استخدمه كمراجع أو طبقة دعم فقط.
- لا توسع إلى أصول كثيرة قبل إتقان الذهب.
- لا تفعل auto-learning المباشر قبل وجود baseline واضح وقياسات مستقرة.
- لا تعقد scale-in قبل إصلاح execution وstate machine.

(17) الخلاصة النهائية

إلى: Stateful SMC Setup Engine + Strategy Profiles + Zone Execution + Contextual Learning.
ليس مجرد تحسين thresholds داخل Consensus Bot، بل تغيير فلسفة النظام من: مؤشرات + إجماع + فلترة،
التحول المطلوب
اليدوي القوي، مع ميزة إضافية لا يملكها معظم المحللين: الانضباط، والقياس، والقدرة على التحسن المستمر.
إذا نفذت هذه الخارطة بالترتيب، فسيقترب النظام كثيرا من منطق المحلل